

Lab 1: RBAC

```
Query  Query History
1  create role cashier_role;
2
3

Data Output  Messages  Notifications
CREATE ROLE
Query returned successfully in 79 msec.
```

```
2
3  create user nam with password 'nam123';
4  create user nhat with password 'nhat123';

Data Output  Messages  Notifications
CREATE ROLE
Query returned successfully in 120 msec.
```

```
5  create user nhat with password 'nhat123';
6
7  grant cashier_role to nam;
8  grant cashier_role to nhat;
9

Data Output  Messages  Notifications
NOTICE: role "nam" has already been granted membership in role "cashier_role" by role "postgres"
NOTICE: role "nhat" has already been granted membership in role "cashier_role" by role "postgres"
GRANT ROLE
Query returned successfully in 65 msec.
```

```
8
9 grant select on film, staff to cashier_role;
10 grant select, insert on payment to cashier_role;
```

Data Output Messages Notifications

GRANT

Query returned successfully in 76 msec.

Lab 2: Implementing RLS

```
13 create policy isolate_cashier_payments on payment
14 for select
15 to cashier_role using (staff_id = (select staff_id from staff where username = current_
16
```

Data Output Messages Notifications

CREATE POLICY

Query returned successfully in 65 msec.

```
17 set role nam;
18 select count(*) from payment;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	count bigint
1	0

```
20  
21 set role nhath;  
22 select count(*) from payment;  
23
```

Data Output Messages Notifications



Showing rows: 1 to 1 Page No: 1 of 1

	count bigint
1	0